Week 4: Building Machine Learning Pipelines on the Urban Data Platform

The municipality now wants to use the data platform developed during the previous weeks for predictive analytics. Rather than answering historical questions, planners want to predict future urban trends and support data-driven decision making. Your task is to build a scalable and reusable machine learning pipeline using Spark MLlib.

The emphasis of this assignment is not on developing sophisticated machine learning models. Instead, it is on demonstrating how a well-designed data platform enables scalable feature engineering, reproducible model training, and maintainable machine learning workflows.

Task 1. Design the Training Dataset

The integrated platform contains information from taxi trips, weather, air quality, and taxi zones. Design a machine learning dataset that can be generated automatically from the integrated platform. Select **one** prediction problem, such as:

- hourly taxi demand,
- trip duration,
- fare amount.

Generate a training dataset containing:

- the target variable,
- feature columns,
- training, validation, and test splits.

The training dataset should be generated automatically from the integrated Delta tables using Spark.

Discuss:

- how the target variable was defined,
- which features were included and why,
- which datasets contributed useful information,
- which assumptions were made during feature construction.

Task 2. Build a Reusable Feature Engineering Pipeline

The integrated dataset produced in Week 1 serves as the input to the machine learning pipeline. Your task is to transform it into a feature dataset suitable for model training. Implement a reusable Spark pipeline that automatically:

- generates temporal features (e.g., hour of day, day of week, month),
- generates location-based features (e.g., pickup zone, dropoff zone, borough),
- encodes categorical variables,
- handles missing values,
- scales or normalizes numerical features where appropriate,
- removes unnecessary attributes,
- prepares the final training dataset for machine learning.

The pipeline should be reusable and require only minimal changes when new features or datasets are added.

Discuss

- why each feature was selected,
- which features required the most preprocessing,
- which datasets contributed the most valuable features,
- how the pipeline supports future extensions.

Task 3. Build a Reproducible ML Pipeline

Implement a complete Spark ML pipeline. Your pipeline should:

- load the training dataset,
- apply the feature engineering pipeline developed in Task 2,
- train an appropriate Spark MLlib model for the selected prediction task,
- evaluate the trained model,
- save the trained model,
- support retraining by reusing the pipeline when new training data becomes available.

The objective is to demonstrate a reusable and reproducible workflow rather than maximize prediction accuracy.

Discuss

- Which parts of the pipeline are reusable?
- How can new features be incorporated?
- How is retraining supported?
- How could the pipeline support multiple prediction tasks?

Task 4. Evaluate the Role of Data Engineering

Evaluate how the platform developed during Weeks 1–3 supports machine learning by comparing two different workflows for building the same training dataset.

Approach A. Build the training dataset directly from the raw datasets. Starting from the original input files (Taxi Trips, Weather, Air Quality, and Taxi Zone Lookup), implement all the required data preparation steps yourself, including loading, cleaning, integrating, and feature engineering.

Approach B. Build the training dataset using the integrated platform developed during Weeks 1–3. Start from the integrated Delta tables produced by your data platform. Reuse the ingestion, validation, and integration components developed during Weeks 1–3, then apply the feature engineering pipeline developed in **Task 2** to prepare the final training dataset.

Compare the two approaches in terms of:

- implementation complexity,
- preprocessing complexity,
- training time,
- reproducibility.

Discuss

- Which engineering decisions from Weeks 1–3 simplified machine learning?
- Which datasets contributed the most useful features?
- Which parts of the workflow became reusable?
- How did the integrated platform reduce the amount of preprocessing required before feature engineering?
- How would the ML pipeline change if additional datasets were introduced?
- What improvements to the platform would most improve future machine learning applications?

Support your discussion using your implementation.

Deliverables

Each group should submit

- **Source code**, i.e., the complete Spark project, including the training dataset generation pipeline, reusable feature engineering pipeline, Spark ML pipeline, model evaluation, comparison experiments, and configuration files.
- **A 3–5 page design report**, containing the selected prediction problem, training dataset design, feature engineering strategy, machine learning pipeline, engineering decisions, and trade-offs.
- **A short evaluation report**, including the feature engineering analysis, model evaluation results, comparison of the two development approaches, reproducibility analysis, and discussion of the role of data engineering in supporting machine learning.

- **A README** describing how to generate the training dataset, run the feature engineering pipeline, train and evaluate the model, reproduce the comparison experiments, and retrain the model when new data becomes available.